

Machine Learning for Mechanical Engineers at CoEP Session 14: KNN

Session 14: KNN

KNN

K-Nearest Neighbors (KNN)

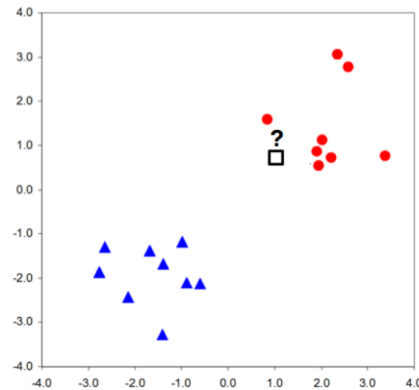
- Supervised Classification.
- Algorithm stores all available cases and classifies new cases by a majority vote of its k neighbors.

K-Nearest Neighbors (KNN)

- “A man is known by the company he keeps”
- KNN can easily be mapped to our real lives. If you want to learn about a person, of whom you have no information, you might like to find out about his close friends and the circles he moves in and gain access to his/her information!

Intuition for kNN

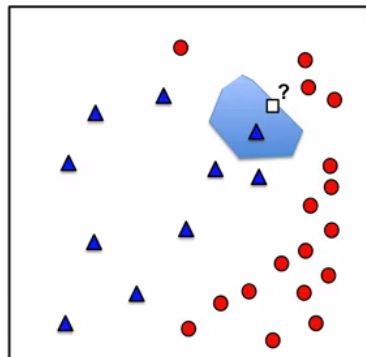
- Set of points (x,y) and two classes (red/blue)
- Is the box red or blue?
- How did you guess? By Decision Tree? or by SVM?
- You saw, nearby points are red.



Neighborhood is the basis of KNN.

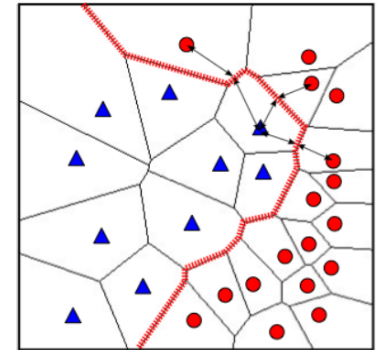
Nearest-neighbor classification

- Use the intuition to classify a new point x: find the most similar training example x'; predict its class y'
- This is true for all the points shown in blue patch/cell.
- Every training example is going to have a small cell around it.
- All cells together are called as Voronoi Diagram.
- Examples: Post office region demarcation, Utility stations, etc.



Nearest-neighbor classification

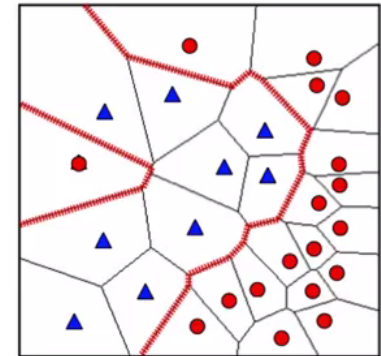
- Each cell is such that points in it are closer to its owner Centroid than any other Centroids.
- Each cell corresponds to a training example.
- Classification boundary separates the cells based on classes
- See how a Non-linear complex partitioning is possible, than just linear or margin separation



Nearest-neighbor classification

Limitations

- But, if you have a Outlier, say the red shown, then whole classification goes for a toss.
- Boundary becomes too complicated.
- So, Sensitive to Outliers.
- Remedy: to look at more than one neighbors to classify yourself.
- Majority Voting. Adds stability.



Scaling Issues

- Attributes may have to be scaled to prevent distance measures from being dominated by one of the attributes
- Example, with three dimensions
 - height of a person may vary from 1.5m to 1.8m
 - weight of a person may vary from 90lb to 300lb
 - income of a person may vary from \$10K to \$1M
- Income will dominate if these variables aren't standardized.

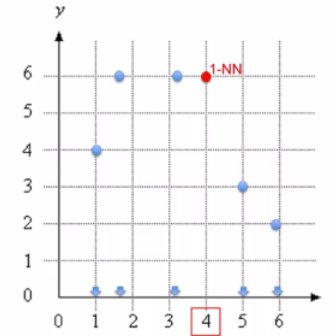
Standardization

- Treat all features “equally” so that one feature doesn't dominate the others
- Common treatment to all variables
 - Standardize each variable: Mean = 0, Standard Deviation = 1
 - Normalize each variable: Max = 1, Min = 0

KNN Algorithm

Classification Algorithm

- Given: training examples (x_i, y_i) and test point x to classify.
- Here, there is no traditional training phase, where model is built.
- Compute distance $D(x, x_i)$ to every training example x_i
- Select k closest instances $x_{i1} \dots x_{ik}$ and their labels $y_{i1} \dots y_{ik}$
- Output the class y^* which is most frequent in $y_{i1} \dots y_{ik}$



KNN as Regression

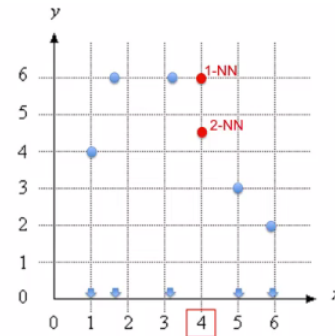
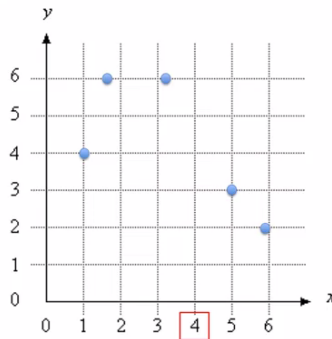
- Given: training examples (x_i, y_i) with y_i as real valued, like profits, price, etc. and test point x to predict the target value.
- Here is no traditional training phase, where model is built.
- Compute distance $D(x, x_i)$ to every training example x_i
- Select k closest instances $x_{i1} \dots x_{ik}$ and their labels $y_{i1} \dots y_{ik}$
- Output the target y^* which is MEAN of $y_{i1} \dots y_{ik}$

KNN Regression Example: 1d

- If we find 2-NN then 3.2 and 5 are the neighbours
- Their y s are 6 and 3.
- Average of those y s is 4.5. So for $x = 4$, y is predicted as 4.5

KNN Regression Example: 1d

- Given: Some set of points (x_i, y_i) . The function/relation between them is unknown.
- Goal: given, unseen x , find its y value.

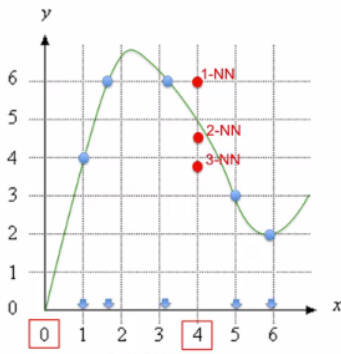


KNN Regression Example: 1d

- Its easy to see that if you go on adding neighbours, you may not be more accurate all the time.
- If you take all the neighbours?
- As it is just average of those many y values around.
- Is that good? No. There is one sweet spot (tradeoff).

KNN Regression Example: 1d

- Who are the neighbours of $x = 4$ on x axis?
- If we want only one nearest neighbour (1-NN) then 3.2 is nearest, its y is 6, so thats the final prediction for $x = 4$.

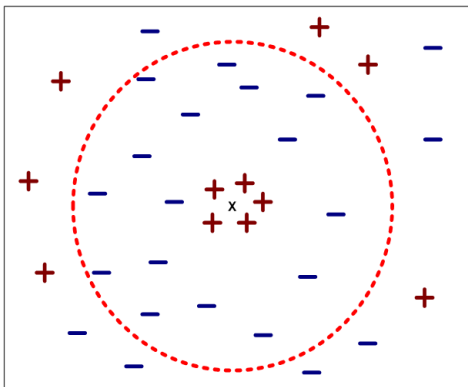


KNN Regression Example: 1d

- So, if you are in the middle, then its interpolation, works.
- If you are at boundaries then, its hard to predict. Extrapolation
- Find for $x = 0$ with 1-NN, 2NN.

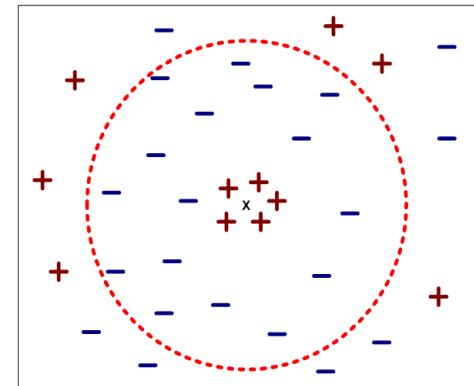
Choosing the right k

- Say, we are classifying middle point, into either +s or -s
- Depending on k (ie number of neighbours), it may get classified as + or -
- Say if k is upto 5, then it would be +, if say, 20 then it would be -ve



Choosing the right k

- If k is too small, sensitive to noise points in the training data
- Susceptible to overfitting
- If k is too large, neighborhood may include points from other classes
- Susceptible to misclassification



Choosing the right k

- Do, cross validation.
- Take validation set out of training set. Make it separate, not part of Training set any more.
- Try different k's on the validation set.
- You will predict y for every x in the test set.
- As you know the correct labels, see which k gives best result.

Similarity Criterion - Distance

Distance measures

- Crucial component
- Finds similarity value between two data points.
- Types: Euclidean (for floats), Hamming (for enums), Minkowski (general form)

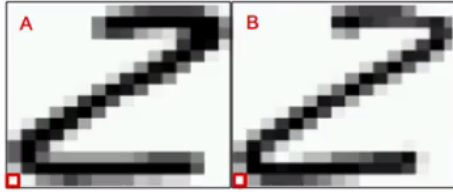
Distances

- The case being assigned to the class is most common among its K nearest neighbors measured by a distance function.
- These distance functions can be Euclidean, Manhattan, Minkowski and Hamming distance.

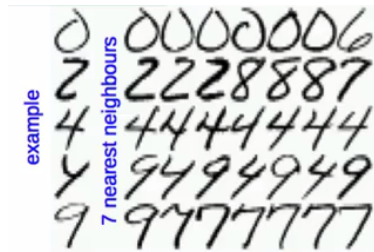
Example

- Handwritten digit recognition
- 16×16 bitmaps
- Euclidean Distance over pixels between testing bitmap A and training bitmap B.

$$D(A, B) = \sqrt{\sum_r \sum_c (A_{r,c} - B_{r,c})^2}$$
- Find 7 nearest training examples, their known digit labels.
- Find most occurring among those labels



Example



- For “0”, max vote goes to “0”.
- For “2”, there is a tie, may add more neighbors to decide.
- For “4”, things may fail.
- Simple algorithm, but impressive results.
- Accuracy:
 - 7-NN: 95.2%
 - SVM: 95.8%
 - Humans: 97.5%, Humans are not perfect either!!

Distance measures (Recap)

- Euclidean is for numerical features: $\sqrt{\sum (x_d - x'_d)^2}$
- Hamming is for categorical features: $\sum 1_{x_d \neq x'_d}$ ie number of attributes, where x, x' differ
- Minkowski distance: $\sqrt[p]{\sum |x_d - x'_d|^p}$
 - $p = 2$: Euclidean
 - $p = 1$: Manhattan
 - $p = 0$: Hamming

Intuition

Minkowski is just one dial (p) that stretches between two familiar rulers: at $p = 1$ you add up the “city block” steps along each axis (Manhattan), while at $p = 2$ you measure the straight-line distance (Euclidean). Larger differences get penalized more heavily as p grows, so the choice of p is really a choice about how harshly you want to punish a single large mismatch versus several small ones.

Implementing KNN

Writing our Own KNN from Scratch

A machine learning algorithm usually consists of 2 main steps:

- A training step that takes as input the training data X and the corresponding target y and outputs a learned model h
- A predict step that takes as input new and unseen observations and uses the function h to output their corresponding responses.

Writing our Own KNN from Scratch

In the case of KNN, which as discussed earlier, is a lazy algorithm, the training block reduces to just memorizing the training data

```
import numpy as np
from collections import Counter

def train(X_train, y_train):
    # do nothing
    return
```

Writing our Own KNN from Scratch

Now we need to write the predict method which must do the following:

- It needs to compute the euclidean distance between the “new” observation and all the data points in the training set.
- It must then select the K nearest ones and perform a majority vote.
- It then assigns the corresponding label to the observation.

Writing our Own KNN from Scratch

```
def predict(X_train, y_train, x_test, k):
    distances = []
    targets = []

    for i in range(len(X_train)):

        distance = np.sqrt(np.sum(np.square(x_test -
                                                X_train[i, :]))

        distances.append([distance, i])

    distances = sorted(distances)

    for i in range(k):
        index = distances[i][1]
        targets.append(y_train[index])

    return Counter(targets).most_common(1)[0][0]
```

Writing our Own KNN from Scratch

Putting it all together, we can define the function `KNearestNeighbor`, which loops over every test example and makes a prediction.

```
def kNearestNeighbor(X_train,y_train,X_test, predictions, k):
    train(X_train, y_train)

    for i in range(len(X_test)):
        predictions.append(predict(X_train, y_train,
                                   X_test[i, :], k))
```

Writing our Own KNN from Scratch

Let's go ahead and run our algorithm

```
from sklearn.metrics import accuracy_score

# making our predictions
predictions = []

kNearestNeighbor(X_train, y_train, X_test, predictions, 7)

# transform the list into an array
predictions = np.asarray(predictions)

# evaluating accuracy
accuracy = accuracy_score(y_test, predictions)
print('\nThe accuracy is %d%%' % (accuracy * 100))
```

Optimizing KNN

- Comparing a query point a in d dimensions against n training examples computes with a runtime of $O(nd)$, which can cause lag as points reach millions or billions.
- Popular choices to speed up KNN include:
 - Venn Diagrams: partitioning plane into regions based on distance to points in a specific subset of the plane
 - Grid Indexes: carve up space into d -dimensional boxes or grids and calculate the NN in the same cell as the point
 - Locality Sensitive Hashing (LSH): abandons the idea of finding the exact nearest neighbors. Instead, batch up nearby points to quickly find the most appropriate bucket B for our query point.

Intuition

All three tricks trade a bit of exactness for speed by pre-organizing the training points so that a query only needs to look at a small nearby subset instead of every single point. Voronoi/Grid indexes still find the true nearest neighbor but narrow the search space first; LSH goes a step further and accepts an approximate answer in exchange for even less work – a reasonable trade when “close enough” neighbors are good enough for the task.

Summary

Final Thoughts on Nearest Neighbors

- Nearest-neighbors classification is part of a more general technique called instance-based learning
- Use specific instances for prediction, rather than a model
- Nearest-neighbors is a lazy learner
- Performing the classification can be relatively computationally expensive
- No model is learned up-front

Final Thoughts on Nearest Neighbors

Pros

- The most attractive features of the K-nearest neighbor algorithm is that is simple to understand and easy to implement.
- With zero to little training time, it can be a useful tool for off-the-bat analysis of some data set you are planning to run more complex algorithms on.
- Furthermore, KNN works just as easily with multiclass data sets whereas other algorithms are hardcoded for the binary setting.
- Finally, as we mentioned earlier, the non-parametric nature of KNN gives it an edge in certain settings where the data may be highly “unusual”.

Final Thoughts on Nearest Neighbors

Cons

- Computationally expensive testing phase which is impractical in industry settings.
- Can suffer from skewed class distributions. For example, if a certain class is very frequent in the training set, it will tend to dominate the majority voting of the new example (large number = more common).
- Finally, the accuracy of KNN can be severely degraded with high-dimension data because there is little difference between the nearest and farthest neighbor.

K-Nearest Neighbors (Recap)

- k = parameter, chosen by analyst
- For a given test instance, use the k “closest” points (nearest neighbors) for performing classification
- “closest” points: defined by some proximity metric, such as Euclidean Distance
- Requires three things
 - The set of stored records
 - Distance Metric to compute distance between records
 - The value of k , the number of nearest neighbors to retrieve

Quick Check: K-Nearest Neighbor

Think About It

Suppose you train a KNN model to predict house prices using square footage (values in the thousands) and number of bedrooms (typically 1-5), without scaling either feature. Which feature will dominate the distance calculation, and why does this make KNN especially sensitive to feature scaling compared to a Decision Tree trained on the same raw data?

Answer

Square footage will dominate, since distances are computed on raw magnitudes and a difference of a few hundred square feet dwarfs a difference of one bedroom. KNN is sensitive to this because every feature contributes directly to a single geometric distance number; a Decision Tree instead considers one feature at a time when choosing splits, so an unscaled feature doesn't drown out the others the way it does for a distance metric.

K-Nearest Neighbors with Scikit-Learn

Knn Template Code

```
from sklearn.neighbors import KNeighborsClassifier

#Assumed you have, X (predictor) and Y (target)
# for training and x_test(predictor) of test

model = KNeighborsClassifier(n_neighbors=6)

model.fit(X, y)

predicted= model.predict(x_test)
```

K-Nearest Neighbors

(Ref: Machine Learning Algorithm Recipes in scikit-learn - Jason Brownlee)

```
# k-Nearest Neighbor
from sklearn import datasets
from sklearn import metrics
from sklearn.neighbors import KNeighborsClassifier
# load iris the datasets
dataset = datasets.load_iris()
# fit a k-nearest neighbor model to the data
model = KNeighborsClassifier()
model.fit(dataset.data, dataset.target)
print(model)
# make predictions
expected = dataset.target
predicted = model.predict(dataset.data)
# summarize the fit of the model
print(metrics.classification_report(expected, predicted))
print(metrics.confusion_matrix(expected, predicted))
```

Test case: Iris Flowers

Review of the iris dataset

```
import pandas as pd

url = 'https://raw.githubusercontent.com/jbrownlee/Datasets/master/iris.csv'

col_names = ['sepal_length', 'sepal_width', 'petal_length',
             'petal_width', 'species']

iris = pd.read_csv(url, header=None, names=col_names)
```

Review of the iris dataset

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

- 150 observations (n=150): each observation is one iris flower
- 4 features: sepal length, sepal width, petal length, petal width
- Response: iris species
- Classification problem since response is categorical

Encoding Iris dataset

Encoding of the Target

```
iris['species_num'] = iris.species.map({'Iris-setosa':0,
                                       'Iris-versicolor':1,
                                       'Iris-virginica':2})
```

Plot Iris dataset

Exploratory Data Analysis

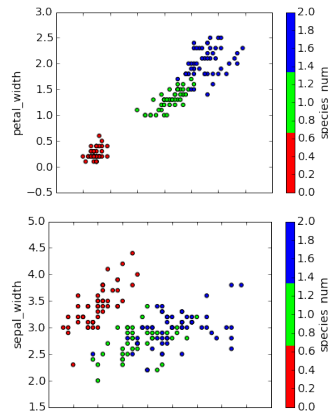
```
# create a scatter plot of PETAL LENGTH versus PETAL WIDTH
# and color by SPECIES
iris.plot(kind='scatter', x='petal_length', y='petal_width', c='species_num',
         colormap=cmap_bold)

# create a scatter plot of SEPAL LENGTH versus SEPAL WIDTH
```

```
# and color by SPECIES
iris.plot(kind='scatter', x='sepal_length', y='sepal_width', c='species_num',
        colormap=cmap_bold)
```

Plot Iris dataset

Exploratory Data Analysis



- setosas have small petals, versicolor have medium sized petals and virginica have the largest petals.
- Furthermore, setosas seem to have shorter and wider sepals than the other two classes.

KNN modeling

```
feature_cols = ['sepal_length', 'sepal_width',
                'petal_length', 'petal_width']

X = iris[feature_cols]
y = iris.species_num

from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=1)

knn.fit(X, y)

knn.predict([[3, 5, 4, 2]]) # 2

X_new = [[3, 5, 4, 2], [5, 4, 3, 2]]
knn.predict(X_new) # 2,1
```

Measuring Error

With $k = 1$

```
from sklearn.model_selection import train_test_split

X_train, X_test, Y_train, Y_test = train_test_split(
    X, y, test_size=0.4, random_state=4)

actual = Y_test

knn = KNeighborsClassifier(n_neighbors=1)
knn.fit(X_train, Y_train)
expected = knn.predict(X_test) #predictions

from sklearn import metrics
score_1 = metrics.accuracy_score(actual, expected)
print(score_1) #0.94
```

Measuring Error

Try range of k values

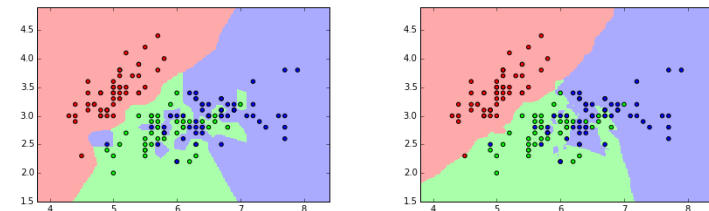
```
import matplotlib.pyplot as plt

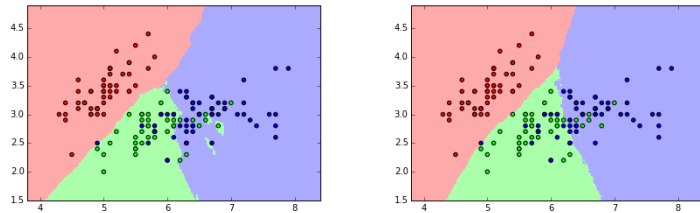
scores = []
k_range = range(1,26)
for k in k_range:
    knn=KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, Y_train)
    actual = knn.predict(X_test)
    scores.append(metrics.accuracy_score(Y_test, actual))

plt.xlabel('Value of K for KNN')
plt.ylabel('Testing Accuracy')
plt.plot(k_range, scores)
plt.show()
```

Effect of K

K=1,5, 15,50





Test case: NBA Players

Exercise

A look at the data, Each row in the data contains information on how a player performed in the 2013-2014 NBA season.

```
import pandas
with open("nba_2013.csv", 'r') as csvfile:
    nba = pandas.read_csv(csvfile)

# The names of all the columns in the data.
print(nba.columns.values)
```

Exercise

Here are some selected columns from the data:

- player - name of the player
- pos - the position of the player
- g - number of games the player was in
- gs - number of games the player started
- pts - total points the player scored

There are many more columns in the data, mostly containing information about average player game performance over the course of the season.

Exercise

Study distance: find the most similar NBA players to LeBron James.

```
import math

# Extract LeBron's row to use as the reference point
selected_player = nba[nba["player"] == "LeBron James"].iloc[0]

# Numeric columns to measure player similarity across stats
distance_columns = ['age', 'g', 'gs', 'mp', 'fg', 'fga',
                    'fg.', 'x3p', 'x3pa', 'x3p.', 'x2p',
                    'x2pa', 'x2p.', 'efg.', 'ft', 'fta',
                    'ft.', 'orb', 'drb', 'trb', 'ast', 'stl',
                    'blk', 'tov', 'pf', 'pts']
```

```
def euclidean_distance(row):
    # Sum squared differences across all stat columns vs LeBron
    inner_value = 0
    for k in distance_columns:
        inner_value += (row[k] - selected_player[k]) ** 2
    return math.sqrt(inner_value)

# Compute distance from every player to LeBron row-by-row
lebron_distance = nba.apply(euclidean_distance, axis=1)
```

Exercise

Normalizing columns

```
# Select only the numeric columns from the NBA dataset
nba_numeric = nba[distance_columns]

# Normalize all of the numeric columns
nba_normalized = (nba_numeric - nba_numeric.mean()) / nba_numeric.std()

# Fill in NA values in nba_normalized
nba_normalized.fillna(0, inplace=True)
```

Exercise

Use the distance.euclidean function from scipy.spatial, a much faster way to calculate euclidean distance.

```
from scipy.spatial import distance

# Find the normalized vector for lebron james.
lebron_normalized = nba_normalized[nba["player"] == "LeBron James"]

# Find the distance between lebron james and everyone else.
euclidean_distances = nba_normalized.apply(lambda row: distance.euclidean(row,
    lebron_normalized), axis=1)

# Create a new dataframe with distances.
distance_frame = pandas.DataFrame(data={"dist": euclidean_distances, "idx":
    euclidean_distances.index})
distance_frame.sort_values("dist", inplace=True)
# Find the most similar player to lebron (the lowest distance to lebron is
    lebron, the second smallest is the most similar non-lebron player)
second_smallest = distance_frame.iloc[1]["idx"]
most_similar_to_lebron = nba.loc[int(second_smallest)]["player"]
```


Exercise

Generating training and testing sets

```
import random
from numpy.random import permutation

# Randomly shuffle the index of nba.
random_indices = permutation(nba.index)

# Set a cutoff for how many items we want in the test set (in this case 1/3 of
  the items)
test_cutoff = math.floor(len(nba)/3)

# Generate the test set by taking the first 1/3 of the randomly shuffled
  indices.
test = nba.loc[random_indices[1:test_cutoff]]

# Generate the train set with the rest of the data.
train = nba.loc[random_indices[test_cutoff:]]
```

Exercise

Using sklearn for k nearest neighbors

```
x_columns = ['age', 'g', 'gs', 'mp', 'fg', 'fga', 'fg.', 'x3p', 'x3pa', 'x3p.',
  'x2p', 'x2pa', 'x2p.', 'efg.', 'ft', 'fta', 'ft.', 'orb', 'drb', 'trb',
  'ast', 'stl', 'blk', 'tov', 'pf']

y_column = ["pts"]

from sklearn.neighbors import KNeighborsRegressor

knn = KNeighborsRegressor(n_neighbors=5)

knn.fit(train[x_columns], train[y_column])
```

```
predictions = knn.predict(test[x_columns])
```

Exercise

Computing error

```
actual = test[y_column]

mse = (((predictions - actual) ** 2).sum()) / len(predictions)
```

Classifier Comparison

Eager Learners

- Decision Trees, SVMs
- Model Building: potentially slow
- Classifying Test Instance: fast
- finding a global model that fits the entire input space

Lazy Learners

- Nearest Neighbors
- Model Building: fast (because there is none!)
- Classifying Test Instance: slow
- classification decisions are made locally (small k values), and are more susceptible to noise

Quick Check: KNN with Scikit-Learn

Think About It

If you evaluate a $k = 1$ KNN model by measuring its accuracy on the very same data it was fit on, it will score close to 100%. Why is this measurement almost meaningless, and how does the “Measuring Error” code above (using `train_test_split`) fix it?

Answer

With $k = 1$, every training point’s single nearest neighbor is itself, so it trivially predicts its own label – the score reflects memorization, not generalization to unseen data. Splitting into train/test sets forces the model to predict labels for points it never saw during fitting, which is the only way to estimate how it will perform on new data.